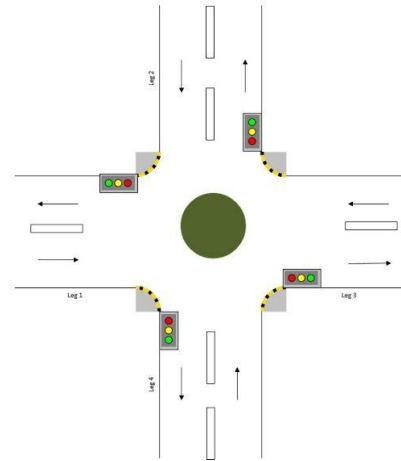


Department of Electronics and Electrical Engineering

Traffic Light System Using Basys3 FPGA



Team 9

Mohammed Osama	55-20087 [T-36]
Styven Hany	55-10067[T-36]
Bassam Walid	55-0499[T-38]
Kareem Hassan	55-0427[T-37]
Ahmed Amr	55-0477[T-33]
Yousef Nader Greiss	55-1290[T-30]
Somaya Magdy	55-5027[T-38]
Sara Hany	55-19029[T-38]
Yousef Mohammed	55-1031 [T-36]
Ali fathy	55- 5880 [t-34]
Mostafa Shakweer	56- 29282 [T-38]

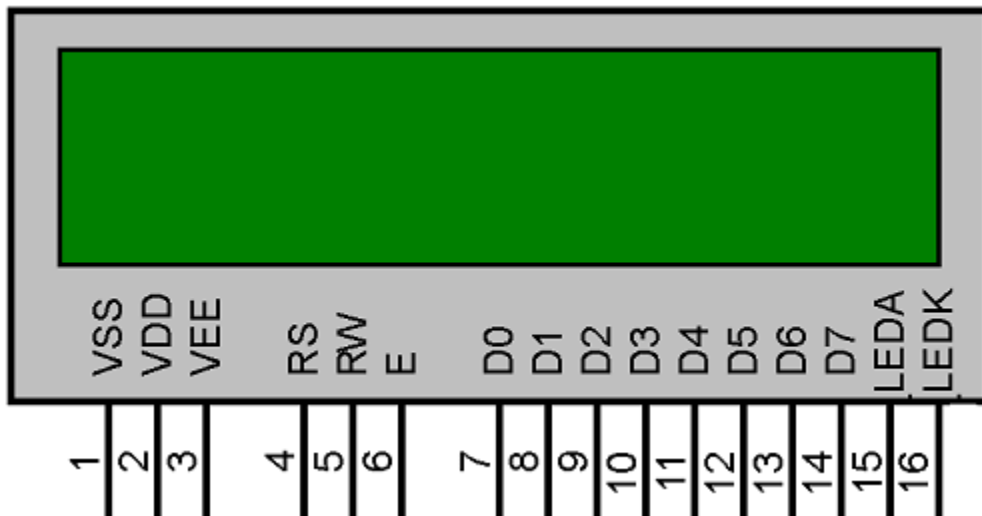
Group representative: Bassam Walid

bassam.aboelhamaid@student.guc.edu.eg

Components Used:

- FPGA Basys3(Hardware)
- LCD 16x2 (for displaying Temperature and Statements)
- LM35 Temperature Sensor
- IR Infrared Sensor
- Buzzer
- 220 Volt Leds
- Wires and relays
- Arduino
- Mocket

LCD 16X2 Datasheet:



Sr. No	Pin No.	Pin Name	Pin Type	Pin Description	Pin Connection
1	Pin 1	Ground	Source Pin	This is a ground pin of LCD	Connected to the ground of the

					MCU/ Power source
2	Pin 2	VCC	Source Pin	This is the supply voltage pin of LCD	Connected to the supply pin of Power source
3	Pin 3	V0/VEE	Control Pin	Adjusts the contrast of the LCD.	Connected to a variable POT that can source 0-5V
4	Pin 4	Register Select	Control Pin	Toggles between Command/Data Register	Connected to a MCU pin and gets either 0 or 1. 0 -> Command Mode 1-> Data Mode
5	Pin 5	Read/Write	Control Pin	Toggles the LCD between Read/Write Operation	Connected to a MCU pin and gets either 0 or 1. 0 -> Write Operation 1-> Read Operation
6	Pin 6	Enable	Control Pin	Must be held high to perform Read/Write Operation	Connected to MCU and always held high.
7	Pin 7-14	Data Bits (0-7)	Data/Command Pin	Pins used to send Command or data to the LCD.	<u>In 4-Wire Mode</u> Only 4 pins (0-3) is connected to MCU

					<u>In 8-Wire Mode</u> All 8 pins(0-7) are connected to MCU
8	Pin 15	LED Positive	LED Pin	Normal LED like operation to illuminate the LCD	Connected to +5V
9	Pin 16	LED Negative	LED Pin	Normal LED like operation to illuminate the LCD connected with GND.	Connected to ground

LCD 16x2 VHDL Code:

```

Traffic.vhd  x  Basys-3-Master.xdc  x  lcd16x2.vhd  x
C:/Users/HP/OneDrive/Desktop/DSD/project_3/project_3.srscs/sources_1/new/lcd16x2.vhd

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4  use IEEE.STD_LOGIC_ARITH.ALL;
5
6
7
8  entity lcd16x2 is
9      Port ( clk : in STD_LOGIC;
10          lcd_rs : out STD_LOGIC;
11          lcd_e  : out STD_LOGIC;
12          lcd_rw : out STD_LOGIC;
13          data  : out STD_LOGIC_VECTOR (7 downto 0)
14      );
15
16  end lcd16x2;

architecture Behavioral of lcd16x2 is
    type arr is array(1 to 38) of std_logic_vector(7 downto 0);
    constant data_rom: arr := (
        X"38", X"0C", X"06", X"01", X"C0",
        X"53", X"4C", X"4F", X"57", X"20", X"44", X"4F", X"57", X"4E", X"2C", X"50", X"4C", X"45", X"41", X"53", X"45",
        X"0D", X"C0",
        X"44", X"52", X"49", X"56", X"45", X"20", X"53", X"41", X"46", X"45", X"4C", X"59", X"20", X"3A", X"29");
    signal en_timing : integer range 0 to 100000;
    signal data_pos  : integer range 1 to 39;
    signal flag: std_logic := '0';
begin
    lcd_rw<='0';
    process(clk)
    begin
        if rising_edge(clk) then
            if (flag = '0') then

                if en_timing<=50000 then --hena han7ot el data then wait
                    en_timing<=en_timing+1;
                    lcd_e<='1';
                    data<=data_rom(data_pos) (7 downto 0);
                elsif en_timing>50000 and en_timing<100000 then --wait abit then input new letter in next cycle
                    en_timing<=en_timing+1;
                    lcd_e<='0';
                elsif en_timing=100000 then --when you rach 100000, go to next letter and start cyole
                    data_pos<=data_pos+1;
                    en_timing<=0;
                end if;
            end if;
        end if;
    end process;
end architecture Behavioral of lcd16x2;

```

```
45         end if;
46
47
48         if (data_pos<=5) then --ma3na 2no haya5od operation
49             lcd_rs<='0';
50         elsif (data_pos>5 and data_pos < 22) then --mn 2vel hena hya5od letters
51             lcd_rs<='1';
52         elsif (data_pos = 22) then
53             lcd_rs <= '0';
54         elsif (data_pos >= 24 and data_pos <39) then
55             lcd_rs <= '1';
56         elsif (data_pos = 39) then
57             flag <= '1';
58         end if;
59     end if;
60 end if;
61
62 end process;
63
64
65
66 end Behavioral;
```

Code of traffic light and sensor:

```

Traffic.vhd  x  Basys-3-Master.xdc  x  lcd16x2.vhd  x
C:/Users/HP/OneDrive/Desktop/DSD/project_3/project_3.srscs/sources_1/new/Traffic.vhd

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.std_logic_arith.ALL;
4  use IEEE.std_logic_unsigned.all;
5
6  entity Traffic is
7      Port (clk, reset, sensor1, sensor2, sensor3 , sensor4 : in std_logic ;
8            G1, Y1, R1, G2, Y2, R2: out std_logic ;
9
10         lcd_rs : out STD_LOGIC;
11         lcd_e : out STD_LOGIC;
12         lcd_rw : out STD_LOGIC;
13         data : out STD_LOGIC_VECTOR (7 downto 0);
14
15         buzzer: out std_logic := '0');
16 end Traffic;
17
18 architecture Behavioral of Traffic is
19     type state_type is (S0, S1, S2, S3, S4, S5, S6);
20     signal S : state_type;
21     signal count1: std_logic_vector(6 downto 0):= "0000000";
22     signal count2: integer := 0;
23     signal count3: integer := 0; -- Sensor1,2 -> red2
24     signal count4: integer := 0;
25
26
27     component lcd16x2 is
28     port(clk : in STD_LOGIC;
29         lcd_rs : out STD_LOGIC;
30         lcd_e : out STD_LOGIC;
31         lcd_rw : out STD_LOGIC;
32         data : out STD_LOGIC_VECTOR (7 downto 0));
33     end component;
34
35     begin
36
37
38     lcd: lcd16x2 port map (clk,lcd_rs,lcd_e,lcd_rw,data);
39

```

```

40
41
42     States : process (clk, reset)
43         begin
44
45             -- reset 3adi gedan byd5lna fel idle state
46             if (reset = '1') then
47                 S <= S0;
48                 count2 <= 100000000;
49                 count1 <= "0000000";
50             -- el code el le lazma
51             elsif (clk = '1' and clk'event) then
52                 --count da by2olena nroo7 lel next light wala la2, 2ma yb2a
53                 --ba 0 yb2a 27na da5lna 5alsna el state el 27na feha
54                 if (count2 = 0) then
55                     count2 <= 100000000;
56                     if (count1 = "0000000") then
57                         case (S) is
58                             when S0 =>
59                                 count1 <= "0000011";
60                                 S <= S1;
61                             -- IDEAL CASE ALL OUTPUTS CAN BE 0
62                             when S1 =>
63                                 count1 <= "01111100";
64                                 S <= S2;
65                             when S2 =>
66                                 count1 <= "0000011";
67                                 S <= S3;
68
69                             when S3 =>
70                                 count1 <= "0000011";
71                                 S <= S4;
72                             when S4 =>
73                                 count1 <= "01111100";
74                                 S <= S5;
75                             when S5 =>
76                                 count1 <= "0000011";
77                                 S <= S6;
78                             when S6 =>
79                                 count1 <= "0000011";
80                                 S <= S1;
81                             when others => S <= S0;
82                         end case;
83                     else
84                         count1 <= count1 - '1';
85                     end if;
86                 else
87                     count2 <= count2 - 1;
88                 end if;
89             end if;
90         end process;
91

```



```

92      Outputs : process(S)
93      Begin
94          case (S) is
95              when S0 =>
96
97                  G1 <= '1';
98                  Y1 <= '1';
99                  R1 <= '1';
100                 G2 <= '1';
101                 Y2 <= '1';
102                 R2 <= '1';
103
104                 -- IDEAL CASE ALL OUTPUTS CAN BE 0
105
106             when S1 =>
107
108                 G1 <= '0';
109                 Y1 <= '1';
110                 R1 <= '0';
111                 G2 <= '0';
112                 Y2 <= '0';
113                 R2 <= '1';
114
115             when S2 =>
116
117                 G1 <= '1';
118                 Y1 <= '0';
119                 R1 <= '0';
120                 G2 <= '0';
121                 Y2 <= '0';
122                 R2 <= '1';
123
124             when S3 =>
125
126                 G1 <= '0';
127                 Y1 <= '1';
128                 R1 <= '0';
129                 G2 <= '0';
130                 Y2 <= '0';
131                 R2 <= '1';
132
133             when S4 =>
134
135                 G1 <= '0';
136                 Y1 <= '0';
137                 R1 <= '1';
138                 G2 <= '0';
139                 Y2 <= '1';
140                 R2 <= '0';
141
142             when S5 =>
143
144                 G1 <= '0';
145                 Y1 <= '0';
146                 R1 <= '1';
147                 G2 <= '1';
148                 Y2 <= '0';
149                 R2 <= '0';

```

```

139         when s6 =>
140             G1 <= '0';
141             Y1 <= '0';
142             R1 <= '1';
143             G2 <= '0';
144             Y2 <= '1';
145             R2 <='0';
146         end case;
147     end process;
148
149     sensor : process(clk)
150     begin
151         if (clk = '1' and clk'event) then --1
152
153
154             if (s=s4 or s=s5 or s=s6) then
155
156                 if (sensor1 = '0' or sensor2 = '0') then --Red2
157                     count4 <= count4 + 1;
158                 end if; --3
159
160             elsif (s=s1 or s=s2 or s=s3) then
161                 if (sensor3 = '0' or sensor4 = '0') then --Red1
162                     count4 <= count4 + 1;
163                 end if;
164
165             else
166                 count3 <= 0;
167                 buzzer <= '0';
168                 count4 <= 0;
169             end if;
170
171             if (count4 > 10000000) then
172                 count4 <= 0;
173                 count3 <= 500000000;
174             end if;
175
176             if (count3 <= 0) then
177                 buzzer <= '0';
178             else
179                 buzzer <= '1';
180                 count3 <= count3 - 1;
181             end if;
182
183         end if; --1
184     end process;
185
186 end Behavioral;

```

الجامعة الألمانية بالقاهرة

Constrain File

```

Traffic.vhd x Basys-3-Master.xdc x lcd16x2.vhd x
C:/Users/HP/Downloads/GUC_122_61_36184_2023-10-09T08_19_13/Basys-3-Master.xdc

1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project
5
6  ## Clock signal
7  set_property PACKAGE_PIN W5 [get_ports clk]
8      set_property IOSTANDARD LVCMOS33 [get_ports clk]
9      create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
10
11 ## Switches
12 set_property PACKAGE_PIN V17 [get_ports {reset}]
13     set_property IOSTANDARD LVCMOS33 [get_ports {reset}]
14 #set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
15 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
16 #set_property PACKAGE_PIN W16 [get_ports {sw[2]}]
17 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
18 #set_property PACKAGE_PIN W17 [get_ports {sw[3]}]
19 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
20 #set_property PACKAGE_PIN W15 [get_ports {sw[4]}]
21 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
22 #set_property PACKAGE_PIN V15 [get_ports {sw[5]}]
23 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
24 #set_property PACKAGE_PIN W14 [get_ports {sw[6]}]
25 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
26 #set_property PACKAGE_PIN W13 [get_ports {sw[7]}]
27 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
28 #set_property PACKAGE_PIN V2 [get_ports {sw[8]}]
29 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[8]}]
30 #set_property PACKAGE_PIN W3 [get_ports {sw[9]}]
...

151
152 ##Pmod Header JB
153 ##Sch name = JB1
154 set_property PACKAGE_PIN A14 [get_ports {data[0]}]
155     set_property IOSTANDARD LVCMOS33 [get_ports {data[0]}]
156 ##Sch name = JB2
157 set_property PACKAGE_PIN A16 [get_ports {data[1]}]
158     set_property IOSTANDARD LVCMOS33 [get_ports {data[1]}]
159 ##Sch name = JB3
160 set_property PACKAGE_PIN B15 [get_ports {data[2]}]
161     set_property IOSTANDARD LVCMOS33 [get_ports {data[2]}]
162 ##Sch name = JB4
163 set_property PACKAGE_PIN B16 [get_ports {data[3]}]
164     set_property IOSTANDARD LVCMOS33 [get_ports {data[3]}]
165 ##Sch name = JB7
166 set_property PACKAGE_PIN A15 [get_ports {data[4]}]
167     set_property IOSTANDARD LVCMOS33 [get_ports {data[4]}]
168 ##Sch name = JB8
169 set_property PACKAGE_PIN A17 [get_ports {data[5]}]
170     set_property IOSTANDARD LVCMOS33 [get_ports {data[5]}]
171 ##Sch name = JB9
172 set_property PACKAGE_PIN C15 [get_ports {data[6]}]
173     set_property IOSTANDARD LVCMOS33 [get_ports {data[6]}]
174 ##Sch name = JB10
175 set_property PACKAGE_PIN C16 [get_ports {data[7]}]
176     set_property IOSTANDARD LVCMOS33 [get_ports {data[7]}]
177
178
179

```

```

209  ##Pmod Header JXADC
210  ##Sch name = XA1_P
211  set_property PACKAGE_PIN J3 [get_ports {G1}]
212      set_property IOSTANDARD LVCMOS33 [get_ports {G1}]
213  ##Sch name = XA2_P
214  set_property PACKAGE_PIN L3 [get_ports {Y1}]
215      set_property IOSTANDARD LVCMOS33 [get_ports {Y1}]
216  ##Sch name = XA3_P
217  set_property PACKAGE_PIN M2 [get_ports {R1}]
218      set_property IOSTANDARD LVCMOS33 [get_ports {R1}]
219  ##Sch name = XA4_P
220  set_property PACKAGE_PIN N2 [get_ports {G2}]
221      set_property IOSTANDARD LVCMOS33 [get_ports {G2}]
222  ##Sch name = XA1_N
223  set_property PACKAGE_PIN K3 [get_ports {Y2}]
224      set_property IOSTANDARD LVCMOS33 [get_ports {Y2}]
225  ##Sch name = XA2_N
226  set_property PACKAGE_PIN M3 [get_ports {R2}]
227      set_property IOSTANDARD LVCMOS33 [get_ports {R2}]
228  ##Sch name = XA3_N
229  set_property PACKAGE_PIN M1 [get_ports {buzzer}]
230      set_property IOSTANDARD LVCMOS33 [get_ports {buzzer}]
231  ##Sch name = XA4_N
232  #set_property PACKAGE_PIN N1 [get_ports {buzzer2}]
233  #  set_property IOSTANDARD LVCMOS33 [get_ports {buzzer2}]
234
235
236

```

Test Bench:

Note: we decreased the time in the traffic code since time is too long to run the simulation.

```

Test_behav.wcfg* x Traffic.vhd x lcd16x2.vhd x Test.vhd x
C:/Users/User/OneDrive/Desktop/Traffic/Traffic.srcs/sources_1/new/Test.vhd

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.std_logic_arith.ALL;
4  use IEEE.std_logic_unsigned.all;
5
6  entity Test is
7  -- Port ( );
8  end Test;
9
10 architecture Behavioral of Test is
11 component Traffic
12 Port (clk, reset, sensor1, sensor2, sensor3, sensor4 : in std_logic ;
13       G1, Y1, R1, G2, Y2, R2: out std_logic ;
14
15       lcd_rs : out STD_LOGIC;
16       lcd_e : out STD_LOGIC;
17       lcd_rw : out STD_LOGIC;
18       data : out STD_LOGIC_VECTOR (7 downto 0);
19
20       buzzer: out std_logic := '0');
21 end component;
22 signal clk: std_logic:='0';
23 signal reset: std_logic:='0';
24
25 signal y1: std_logic;
26 signal r1: std_logic;
27 signal g1: std_logic;
28 signal y2: std_logic;
29 signal r2: std_logic;
30 signal g2: std_logic;
31
32 signal buzzer: std_logic;
33
34 signal sensor1: std_logic;
35 signal sensor2: std_logic;
36 signal sensor3: std_logic;
37 signal sensor4: std_logic;
38
39 signal lcd_rs : STD_LOGIC; --law ba 0 yb2a ht3mel operation law ba 1 yb2a hatda5al letter
40 signal lcd_e : STD_LOGIC;
41 signal lcd_rw : STD_LOGIC;
42 signal data : STD_LOGIC_VECTOR (7 downto 0);
43
44 begin
45 uut:Traffic port map (clk,reset,sensor1,sensor2,sensor3,sensor4,g1,y1,r1,g2,y2,r2,lcd_rs,lcd_e,lcd_rw,data,buzzer);
46
47 clk_process: process
48 begin
49   clk<='0';
50   wait for 0.5ns;
51   clk<='1';
52   wait for 0.5ns;
53 end process;
54
55 stim_proc:process
56 begin
57   reset<='1';
58   wait for 0.5ns;
59   reset<='0';
60   sensor1 <= '1';
61   sensor2 <= '1';

```

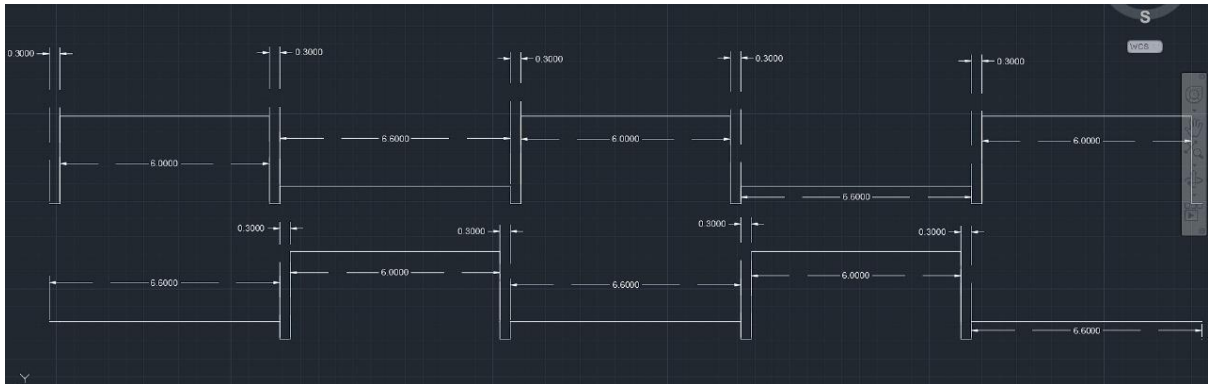
```

61  ○ sensor2 <= '1';
62  ○ sensor3 <= '1';
63  ○ sensor4 <= '1';
64  ○ wait for 1000ns;
65  ○ sensor1 <= '0';
66  ○ sensor2 <= '1';
67  ○ sensor3 <= '1';
68  ○ sensor4 <= '1';
69  ○ wait for 1000ns;
70  ○ sensor1 <= '1';
71  ○ sensor2 <= '0';
72  ○ sensor3 <= '1';
73  ○ sensor4 <= '1';
74  ○ wait for 1000ns;
75  ○ sensor1 <= '1';
76  ○ sensor2 <= '1';
77  ○ sensor3 <= '0';
78  ○ sensor4 <= '1';
79  ○ wait for 1000ns;
80  ○ sensor1 <= '1';
81  ○ sensor2 <= '1';
82  ○ sensor3 <= '1';
83  ○ sensor4 <= '0';
84  ○ wait for 1000ns;
85  ○ sensor1 <= '1';
86  ○ sensor2 <= '1';
87  ○ sensor3 <= '1';
88  ○ sensor4 <= '1';
89  ○ wait for 6000ns;
90  ○ sensor1 <= '0';
91  ○ sensor2 <= '1';

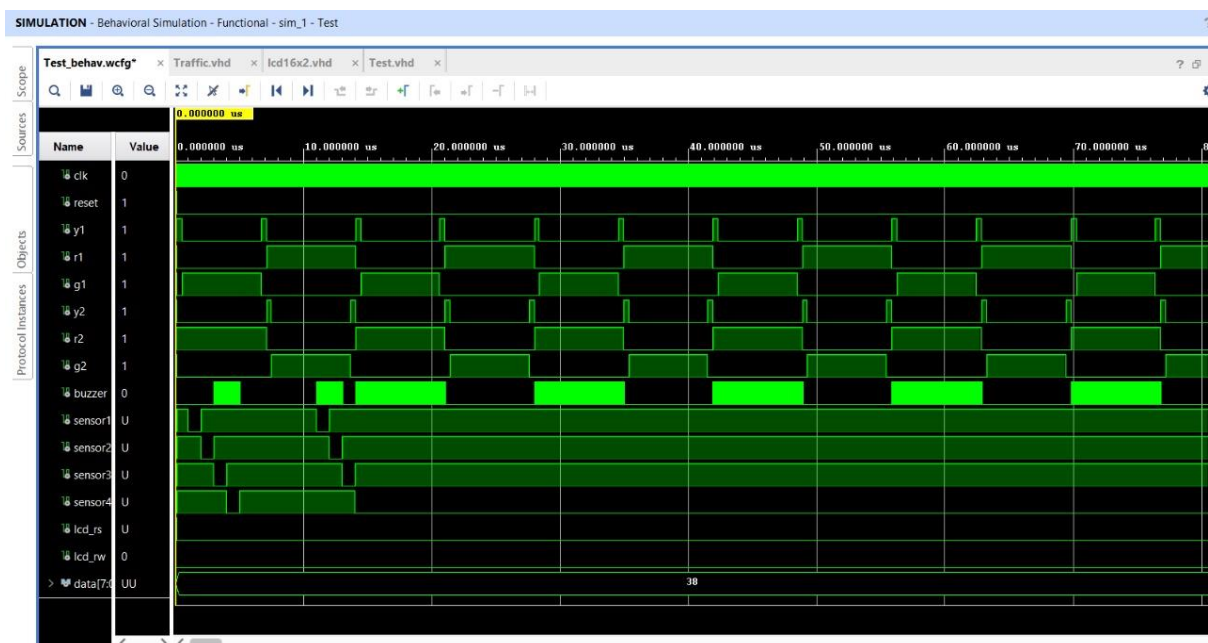
91  ○ sensor2 <= '1';
92  ○ sensor3 <= '1';
93  ○ sensor4 <= '1';
94  ○ wait for 1000ns;
95  ○ sensor1 <= '1';
96  ○ sensor2 <= '0';
97  ○ sensor3 <= '1';
98  ○ sensor4 <= '1';
99  ○ wait for 1000ns;
100 ○ sensor1 <= '1';
101 ○ sensor2 <= '1';
102 ○ sensor3 <= '0';
103 ○ sensor4 <= '1';
104 ○ wait for 1000ns;
105 ○ sensor1 <= '1';
106 ○ sensor2 <= '1';
107 ○ sensor3 <= '1';
108 ○ sensor4 <= '0';
109 ○ wait for 1000ns;
110 ○ sensor1 <= '1';
111 ○ sensor2 <= '1';
112 ○ sensor3 <= '1';
113 ○ sensor4 <= '0';
114 ○ wait;
115
116 ⊞ end process;
117
118 ⊞ end Behavioral;

```

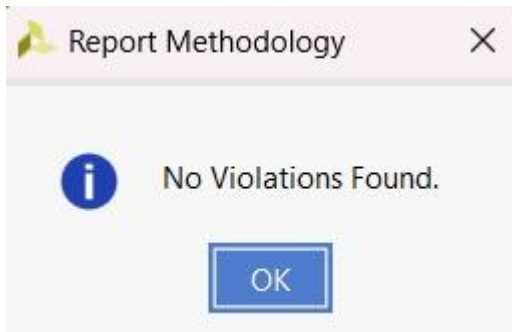
Simulation:



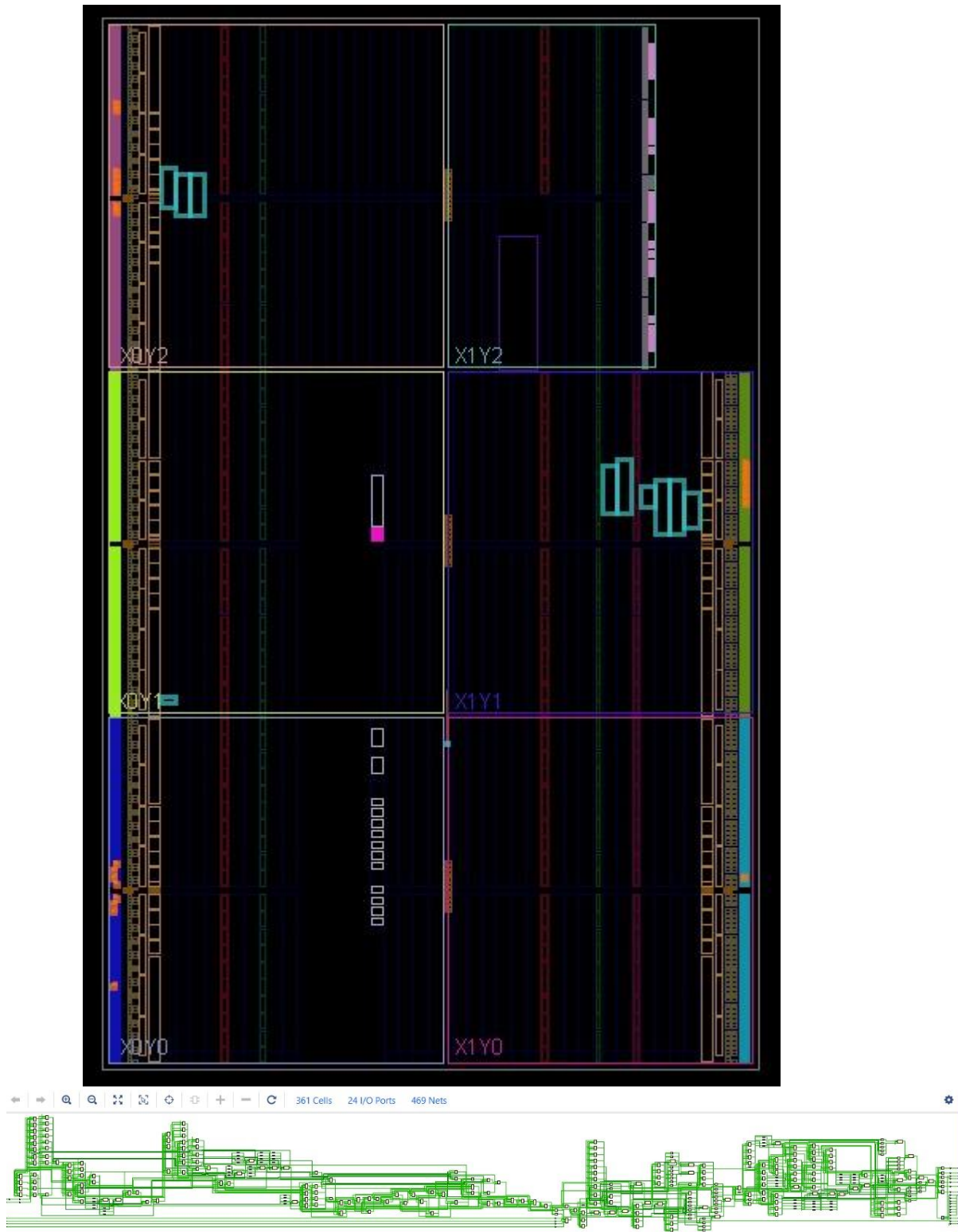
This is a timing diagram representing the system. The 3 second dash represents the yellow light. The 60 seconds represents the green and the 66 seconds represents the red.

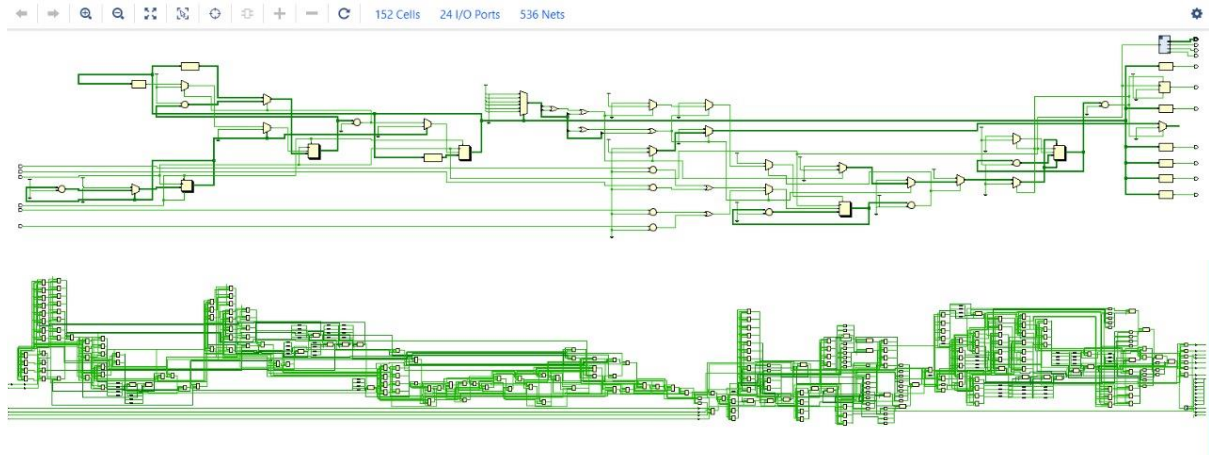


Methodology:



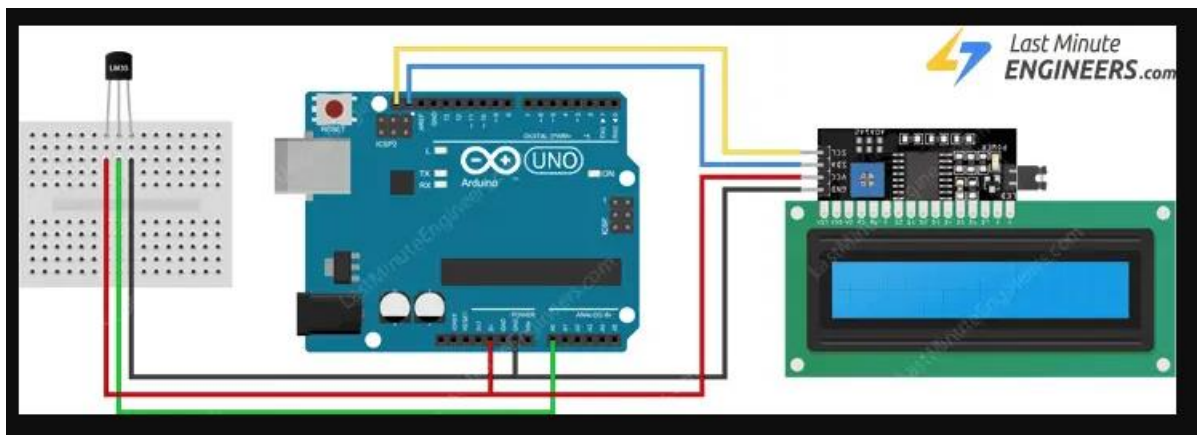
Implementation:





Temperature Sensor LM35 Using Arduino:

- It was very complicated to implement LM35 on FPGA so we used the Arduino.
- Jxadc would not accept analog signals we even tried to use digital analog temperature sensor but faced a problem which was the sensor send 12 bits through one wire with time delay .



```
// Include the LiquidCrystal library
```

```
#include <LiquidCrystal.h>
```

```
// Create a new instance of the LiquidCrystal class
```

```
LiquidCrystal lcd(12, 11, 5, 4, 3, 2); // Use your LCD pin configuration
```

```
// Define a custom degree character
```

```
byte Degree[] = {
```

B00111,

B00101,

B00111,

B00000,

B00000,

B00000,

B00000,

B00000

};

// Define the analog pin, the LM35's Vout pin is connected to

#define sensorPin A2

void setup() {

// Set up the LCD's number of columns and rows

lcd.begin(16, 2);

// Create a custom character

lcd.createChar(0, Degree);

}

void loop() {

// Get the voltage reading from the LM35

int reading = analogRead(sensorPin);

// Convert the reading to voltage

float voltage = (reading / 1024.0) * 5000.0;

// Convert voltage to temperature in Celsius

```
float temperatureC = voltage / 10.0;
```

```
// Print the temperature on the LCD;
```

```
lcd.clear(); // Clear the LCD before printing new values
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("Temperature:");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print(temperatureC, 1);
```

```
lcd.write((uint8_t)0); // print the custom degree character
```

```
lcd.print("C ");
```

```
// Convert Celsius to Fahrenheit
```

```
float temperatureF = (temperatureC * 9.0 / 5.0) + 32.0;
```

```
lcd.print(temperatureF, 1);
```

```
lcd.write((uint8_t)0); // print the custom degree character
```

```
lcd.print("F ");
```

```
delay(1000); // wait a second between readings
```

```
}
```

- 1) When there is a change in the state, we found out that there was an issue regarding the current that made the IR sensor turn off which gave the FPGA an input of '0' and this made the buzzer turn on. We solved this problem by creating a counter (count4) that increments once the state changes and when the counter reaches 1/10 second the buzzer is given input 1 and turns the buzzer on.
- 2) A challenge that we had was to make the pedestrian crossing lights work with the code, we resolved this problem by giving the pedestrian light input from the main traffic light. Solution was that, for Red pedestrian 1 (Rp1) we used the output of inverse of Red 1 (R1') and for Gp1 we used the output of R2'.
- 3) Another problem was the feedback current. We tried to put more than one wire that have another function in series with each other to "OR", but feedback occurred so we used solution 2 to avoid it.
- 4) Vivado couldn't run the test bench simulation with the original timing as they were too large so we edited the original code with smaller timings for the behavioural simulation. You will find 2 codes attached in the google drive. One that runs the system and one that runs the test bench.